

AI时代，我们为什么还要学习编程？

在正式开始学习 C 语言之前，我想先讨论一个可能很多人都想过的问题：

现在 AI 已经可以写代码了，我们为什么还要学习编程？

今天我们可以把一段需求直接交给 AI：

“帮我写一个 PID 控制程序。” “帮我生成一个摄像头循迹算法。” “帮我写一个电机驱动。” “这个程序报错了，帮我修改一下。”

几秒钟之后，AI 就可能给出一份看起来非常完整的代码。

于是一个很自然的问题就出现了：

既然 AI 已经能写代码，那么以后我们是不是只需要学会使用 AI，而不需要再学习编程？

答案恰恰相反。

AI 越强，我们越需要理解编程、算法和工程。

因为真正需要学习的，正在从：

“如何把代码写出来”

逐渐转变为：

“如何判断这段代码到底对不对。”

1. 先对 AI 祛魅：它并不是“什么都知道”

现在的大语言模型，本质上在做的一件事是：

根据前面的内容，预测接下来最可能出现什么。

它学到的不是“事实”，而是大量隐藏在语言里的规律。所以它往往能给出“很合理”的答案——但“看起来合理”和“实际正确”，是两件完全不同的事。

更危险的是，AI 可以非常流畅地说出错误的东西：结构完整、逻辑顺畅、术语专业，甚至带着代码和公式，但实际上是错的。人类不知道时会说“我不知道”，AI 有时候不会。

我们项目里就真实发生过这样一件事：识别模块的函数写得明明白白，可在另一个头文件里，藏着这样一行：

```
#define APP_RECOGNIZE_USE_CLEAR // 让识别模块“闭嘴”，防止链接冲突
```

识别源码用这行宏来决定要不要编译主体。结果，那几百行识别逻辑虽然写在文件里，却被预处理器整体排除——链接时直接报“函数未定义”。后来删掉这行失效的宏，模块才重新接回来。

AI 如果只看那个识别函数，可能认真检查几百行状态机，却漏掉另一个文件里、决定这几百行是否生效的一行代码。它给你的“完整代码”，可能并不完整。要看出这种问题，你得懂头文件、宏、条件编译和链接。

所以用 AI 的第一条原则是：

“看起来合理”不代表“实际正确”，工程里真正可信的，必须经过实验和数据去验证。

2. AI 给方案很容易，做选择才难

假设问 AI “智能车应该用什么循迹方案”，它能一口气列出电磁、摄像头、灰度、摄像头 + PID、摄像头 + 前馈.....这些方案可能都没错。

但问题从来不是“有没有方案”，而是“**哪一个最合适**”。

因为真实工程里的“最优”，取决于一大堆具体条件：赛道什么样、车速多少、用什么 MCU、传感器、规则、成本、功耗、可靠性。

一辆跑 1m/s 的车和一辆跑 4m/s 的车，面对的是完全不同的控制问题；不同的单片机，能承担的计算量也完全不同。

我们组别比赛中就存在着三个看起来相似，实际完全不同的目标：

- 让算法**尽快给出答案**（缩短解算时间）；
- 让小车**走最少的格子**（缩短路径）；
- 让小车**尽快完成整场任务**（缩短真实耗时）。

少走几格，却要多次停站、换方向、校准，可能反而更慢；推完最后一个箱子时离发车区很远，返库时间也会被拖长。旧版本为了“启动快”，直接返回一个贪心解、跳过全局优化，结果就是算得快、跑得慢。

如果只给 AI 一个局部指标，它能把这个指标做得很好；但“这是不是最终目标”，仍要你自己判断。

所以工程不是在找“看起来最厉害”的方案，而是在约束条件下，找：

合适、可靠、可实现、可验证的方案。

3. 真正要培养的，是判断能力

比如 AI 生成一段 PID：

```
output = Kp * error
        + Ki * integral
        + Kd * derivative;
```

不懂控制系统的人，看到它只会问一句：“能不能用？”

但如果你真正理解代码，你会继续追问：

error 是怎么计算的？
控制周期是多少？
integral 会不会越来越大？
有没有积分饱和？
derivative 对噪声敏不敏感？
输出有没有限幅？
程序执行频率是否稳定？
传感器突然异常怎么办？

这时候，你就已经从“写程序”，进入到了“做工程”。

还有一种更隐蔽的错，连测试都会跟着一起错。在实际的比赛过程中，赛规往往会经历多轮完善，而这些细节的改变完全影响工程的设计。你给ai的输入不全面或者有错误，直接导致代码偏离任务预期。如我，组别中旧代码把“箱子不能穿过其他未使用的目标点”当成规则，而测试也接受了这个前提——穿过目标点的路线，被要求返回“无解”。于是代码和测试互相印证，一路“测试通过”。

直到后来对照真实赛规才发现，这个前提本身就是错的，测试断言被反过来改掉：原来的“必须无解”，改成了“必须有解”。

测试通过，只能说明实现满足了测试里的预期；而预期本身对不对，需要赛规、现场行为或独立推导来核对。 错误规则 → 符合错误规则的代码 → 符合错误规则的测试，一样能全绿。所以“验证”这件事，最终也离不开人的知识。

4. 从“写代码”到“做工程”

我们不是要拒绝 AI，恰恰相反，要好好用它——查语法、解释代码、生成简单函数、分析报错、找 Bug、读芯片手册，它都是好帮手。

但记住一句话：

工具越强，对使用者本身的判断能力要求反而越高。

因为真正困难的，从来不是“把代码写出来”，而是“解决一个真实问题”。它是一个循环：

发现问题 → 建立模型 → 拆解问题 → 设计方案 → 编写程序 → 测试 → 发现问题 → 修改方案 →

代码只是这个循环里的一环。随着系统变大，我们要解决的也不再是“如何写出程序”，而是“如何控制程序的复杂度”——所以有了模块化、分层、接口、封装、版本管理、文档、团队协作。

优秀的软件不只是“能跑”，而是出问题时你知道它为什么坏，要改时你知道该改哪里，换个人接手他也能看懂。

5. AI 负责产生可能性，你负责定义“好”

过去学编程，重心在 Coding（怎么写代码）；AI 时代，重心会越来越移向 Problem Solving（怎么解决问题）。

可以概括成一句话：

AI 负责快速产生可能性，人负责定义什么才是“好”。

AI 能一分钟给你几十种方案，但最后总得有人知道：哪一种值得真正装到车上。

所以养成一个习惯，不要只问“帮我写一个程序”，而要学会问：

“为什么这样设计？”“还有哪些方案？”“它有什么缺点？”“什么情况下会失效？”“怎么验证它是对的？”

当你开始这样提问，你才从“AI 的使用者”，变成了“AI 的驾驭者”。

把前面三个真实的坑连起来，其实就是一句话：

AI 能加快代码产生的速度；学习 C，才让我们有能力判断——哪些代码真正运行了、它优化的究竟是什么、它给出的“正确证明”是否可信。

最后

所以今天我们学习 C 语言，并不是因为：

以后所有代码都必须自己一个字符一个字符敲出来。

而是因为 C 语言会帮助我们建立一种最基础的软件思维。

理解：

数据是如何存储的
程序是如何运行的
函数是如何调用的
内存是如何组织的
变量是如何变化的
硬件是如何被软件控制的

只有理解这些之后，

当 AI 给你生成几百行代码时，

你才能真正知道：

它到底写了什么。

也才能在你的小车高速冲向赛道边缘的时候，

不是对着 AI 问一句：

“为什么我的车飞出去了？”

而是知道应该修改哪一个模块，哪一个参数。

一步一步找到问题。

反过来，如果你从一开始就没有去理解这些，情况会完全不一样。

你的整个工程，对你来说，就变成了一个黑箱：

输入（需求、报错信息）



黑箱：一堆你看不懂的代码



输出（能跑 / 不能跑）

你只知道两件事：你喂进去什么，它吐出来什么。

中间到底发生了什么，你一无所知。

于是当小车冲出赛道、程序报错的时候，你唯一能做的，就只剩下把报错原样复制给 AI：

“帮我改一下。”

改完还是不对，再复制给 AI，再改；再不对，再改.....

这会变成一个没有终点的循环：

出问题



复制给 AI



AI 修改



还是不对？



复制给 AI



.....（循环）

因为你自己始终无法判断：

- 这次修改到底对不对；
- 它是不是在用一个新的错误，去掩盖旧的错误；
- 问题到底出在算法、参数、硬件，还是时序。

于是，你就从“工程的主人”，变成了“AI的搬运工”。

而这，恰恰是我们要通过学习编程去避免的。

这，就是我们学习编程真正的意义。

不要把 AI 当成答案。把它当成工具。

不要只学习如何写代码。要学习如何解决一个真实的工程问题。

而 C 语言，就是我们今天迈出的第一步。

也许未来我们亲手写的代码会越来越少，但我们需要理解的系统不会越来越简单。

AI 能替我们完成越来越多的“怎么做”，但至少在可预见的未来，我们仍然需要回答“做什么”“为什么做”和“什么才算做好”。

学习资源：到哪里找答案

学编程、用 AI，遇到问题、卡住、查不到，都是常态。知道去哪里找答案，比背住答案更重要。下面按“编程学习”和“AI 使用”两条线，给你一组渠道。

1. 编程学习

电脑使用入门

不少同学是上大学才第一次真正接触电脑，这很正常——先把电脑用顺手，再谈写代码。

- [Win11 开荒指南（新电脑怎么设置）](#) —— 新买的笔记本要怎么做初始设置、装软件、避开常见坑。
- [Windows 系统 0 基础教程（从开关机学起）](#) —— 完全没接触过电脑也能看懂的系统操作入门。
- [【准大一电脑小白】七天入门电脑基础知识](#) —— 专为准大学生做的系列教程，从零开始。

入门教程

- [菜鸟教程](#) —— C 语言等各语言的语法速查，中文、免费、上手快。
- [廖雪峰的官方网站](#) —— Python、Git、JavaScript 等经典中文教程。
- [中国大学 MOOC](#) —— 名校的系统课程。

视频教程

- [智能车制作开题导论](#) —— 这是一个系列的课，可以通过这个简单了解一下
- [黑马程序员 C 语言零基础入门到精通](#) —— C 语言全套视频教程，零基础可看。

系统学习路线

- [计算机自学指南（CSDIY）](#) —— 系统化的计算机自学路线图，开源、持续更新。
- [计算机教育中缺失的一课](#) —— MIT 经典课程中文版，专讲“学校不教但必备”的东西：命令行、Git、编辑器、脚本。

代码托管

- [GitHub](#) —— 全球最大的开源平台，看别人的代码、找现成的方案。（不要重复造轮子，做之前可以先看有没有类似项目）
- [Gitee](#) —— 国内的代码托管平台，访问更快。

刷题与算法

- [洛谷](#) —— 算法竞赛入门刷题。
- [LeetCode](#) —— 算法与面试刷题。

问答与社区

- [CSDN](#) —— 中文技术博客，遇到具体问题优先搜这里。
- [Stack Overflow](#) —— 全球最大的编程问答社区（英文）。
- [稀土掘金](#) —— 中文技术社区，文章质量不错。
- [B站](#) —— 搜“CH32V307”“智能车”“C 语言”，有大量视频教程。
- [知乎](#) —— 深度经验分享和踩坑总结。

2. AI 使用

把 AI 当成一个“随时可问、不知疲倦的助教”。

常用工具

- [Claude](#)
- [ChatGPT](#)
- [DeepSeek](#)
- [GLM（智谱清言）](#)
- [Kimi](#)
- [Gemini](#)
- [通义千问](#)
- [豆包](#)

想比较各模型当下的实力、选一个顺手的，可以参考这个榜单：[大模型 Agent 榜单](#)。

怎么用

- 查语法、解释代码、分析报错、生成简单函数、读手册，都可以交给它。
- 但记住前言里反复强调的：**带着判断力去用**，多问“为什么这样设计”“怎么验证它是对的”。
- 想让提问更高效，读一遍《提问的智慧》（How-To-Ask-Questions-The-Smart-Way，GitHub 上有中文翻译版）。

前面聊的是“为什么要学”。下面，我们正式开始“到底学什么、怎么上手”。

第一部分：认识嵌入式系统

1. 全国大学生智能汽车竞赛

我们参加的，是**全国大学生智能汽车竞赛**——一项以“车”为载体、融合自动控制、传感器、图像处理等技术的学科竞赛。

几个关键点：

- 分为**竞速类**和**综合创意类**两大类比赛；
- 竞速类又按**缩微赛道**、**室内赛道**、**室外赛道**三个子类设置组别；

- 以**团队参赛**，常规组别每队一般不超过 3 人；
- 晋级路径一般是**校内选拔** → **分赛区** → **全国赛**。

我们实验室参与的，主要是竞速类相关组别。具体组别和规则以当届竞赛官网为准，也可以关注清华的卓晴老师（竞赛技术负责人）的 CSDN 博客了解最新通知：

- [第 21 届智能车竞赛通知（卓晴 · CSDN）](#)

2. 什么是嵌入式系统

在动手之前，先搞清楚一个最基本的问题：

我们写的程序，到底是跑在什么地方的？

简单来说，嵌入式系统就是**藏在设备内部、专门完成特定功能的计算机**。

嵌入式系统：藏在设备内部、专门干一件事的计算机。

它和我们天天用的电脑、手机不一样：

- 功能专一——不是为了“通用”，而是为了“专用”；
- 软硬件都可以裁剪——需要什么就保留什么；
- 对功耗、体积、成本、实时性都有严格的约束。

你手上这块 CH32V307 开发板，就是这样一个系统的核心：一颗单片机（MCU）。

开发板 / 核心板：把单片机做成方便插线、烧程序的板子。**单片机（MCU）**：一块芯片里集成了 CPU、内存和外设，是智能车的“大脑”。

做嵌入式开发，一般会经历这样一个流程：

需求分析 → 选型 → 编码 → 烧录 → 调试 → 迭代

这也正是本次培训“编码 → 编译 → 烧录 → 观察”的实操环节所对应的完整流程。

烧录：把编译好的程序“灌”进单片机。

3. 一辆智能车，就是一个嵌入式系统

把我们最熟悉的智能车拆开来看：

传感器输入 → 控制器运算 → 执行器输出
（摄像头、 （单片机读取 （电机、舵机、
编码器...） 数据并计算） LED...）

- **传感器**负责“感知”：摄像头看路，编码器测转速；
- **控制器**负责“思考”：单片机读取数据、算出一套控制量；
- **执行器**负责“行动”：电机转动、舵机转向、LED 点亮。

以后我们写的每一段代码，都位于“控制器”这一环——它接收输入、做判断、给出输出。

传感器 / 控制器 / 执行器：智能车的“感知 / 思考 / 动作”三段结构——把物理量变成电信号、算出控制量、再驱动机械动作。

4. 软件，是怎么“摸”到硬件的

这里有一个贯穿整个嵌入式开发的核心概念：

对外设的控制，本质上就是对某个特定地址的读写。

一行 C 代码，最终会变成对某个寄存器地址的一次写操作：

一行 C 代码 → 写寄存器 → 引脚电平变化 → LED 亮

后面第三部分你会看到，所谓的“库”，就是把这种对地址的读写，包装成一个个好记、好用的函数。现在只需要记住这一句话即可。

外设：芯片上除 CPU 之外的功能模块，比如 GPIO、串口、定时器。**寄存器：**单片机里“一个地址对应一个开关”的东西，软件控制硬件全靠它。

在进入具体外设之前，还有两个前置事实需要先知道：

- **外设使用前，要先使能它的时钟**——芯片里的每个外设默认是“关闭”的，得先打开它的时钟开关，它才开始工作。
- **一个引脚可能承担多种功能**——同一个物理引脚，既能当普通 GPIO 用，也能复用成 UART、SPI、定时器等外设的引脚（这就是后面会提到的“引脚复用/AFIO”）。

使能时钟：外设默认处于“断电”状态，要先打开它的时钟开关，它才会开始工作。**引脚复用（AFIO）：**同一个引脚可以承担多种功能，按需要切换成 GPIO、串口或定时器的引脚。

第二部分：C 语言入门

这一部分快速过一遍 C 语言的基础语法。目标不是成为语言专家，而是**能读懂、能写出我们工程里会用到的那部分 C 代码**。

不用追求把每个语法点都背下来。看不懂的，随时翻回来查，或者直接问 AI。

1. 程序结构与 main 函数

一个最简单的 C 程序长这样：

```
#include <stdio.h>

int main()
{
    printf("Hello, World!\n"); // 打印一行文字
}
```

```
    return 0;
}
```

为什么程序总是从 `main` 函数开始执行？

因为 `main` 是 C 语言约定的**程序入口**。无论你的代码里有几十个函数，程序运行时都从 `main` 开始，再由 `main` 调用其他函数。

在嵌入式里有一点不同：程序通常不会 `return 0` 退出，而是用一个死循环让它**永远运行下去**（后面会讲到）。

2. 基础语法

几条最常用的规则：

- **头文件包含**：`#include <stdio.h>`——在预处理阶段，把某个头文件的内容“复制粘贴”到当前文件。
- **主函数入口**：`int main()`。
- **语句以分号结尾**：每条语句末尾要写 `;`。
- **注释**：单行用 `//`，多行用 `/* ... */`。

3. 变量与类型

变量就是在内存里申请一块空间，用来存数据：

```
int a = 10;           // 整型
float b = 3.14;       // 浮点型
char c = 'A';         // 字符型
```

嵌入式开发中，我们更习惯用**定宽类型**（占多少字节是确定的）：

```
uint8_t  a = 100;      // 无符号 8 位, 0 ~ 255
uint16_t b = 12345;    // 无符号 16 位
uint32_t c = 123456;   // 无符号 32 位
int8_t   d = -100;     // 有符号 8 位, -128 ~ 127
int16_t  e;            // 有符号 16 位
int32_t  f;            // 有符号 32 位
float    g;            // 单精度浮点
double   h;            // 双精度浮点
```

变量的本质，就是一块被起了名字的内存空间；类型决定了这块空间**占多大、怎么解释**。

4. 运算符

- 算术：`+` `-` `*` `/` `%`（取余）
- 关系：`>` `<` `>=` `<=` `==` `!=`
- 逻辑：`&&`（与） `||`（或） `!`（非）
- 赋值：`=`
- 位运算：`&`（按位与） `|`（按位或） `^`（按位异或） `~`（按位取反） `<<`（左移） `>>`（右移）

```
int a = 5, b = 2;
int sum = a + b;
if (a > b && sum < 10)
{
    printf("条件成立\n");
}
```

位运算是嵌入式里特别重要的一类运算——配置外设、控制寄存器，本质上就是对寄存器某几位做“置 1、清 0、取反、移位”：

```
uint8_t a = 0b00001111;
uint8_t b = a & 0b00000011; // 按位与：只保留低两位 → 0b00000011
uint8_t c = a | 0b10000000; // 按位或：把最高位置 1 → 0b10001111
uint8_t d = a << 1;         // 左移一位：相当于乘 2 → 0b00011110
```

5. 判断：if 与 switch

`if-else` 是两分支：

```
int score = 85;
if (score >= 60)
{
    printf("及格\n");
}
else
{
    printf("不及格\n");
}
```

多分支用 `if-else if-else`：

```
if (score >= 90)    printf("优秀\n");
else if (score >= 80) printf("良好\n");
else if (score >= 60) printf("及格\n");
else                printf("不及格\n");
```

当一个变量有多个离散取值时，用 `switch` 更清晰：

```
switch (day)
{
    case 1: printf("星期一\n"); break;
    case 2: printf("星期二\n"); break;
```

```
default: printf("其他\n"); break;
}
```

`switch` 只能用于整型或枚举；每个 `case` 后要加 `break`，否则会“穿透”继续执行后面的分支。

6. 循环：for 与 while

`for` 循环，用于重复固定次数：

```
for (int i = 0; i < 5; i++)
{
    printf("i = %d\n", i);
}
```

结构：`for` (初始化；条件；更新)。

`while` 循环，只要条件为真就一直执行：

```
while (i < 5)
{
    printf("i = %d\n", i);
    i++;
}
```

死循环在嵌入式里非常常见——程序跑起来之后就不应该停：

```
while (1)
{
    // 一直执行
}
// 等价于
for (;;)
{
    // 一直执行
}
```

7. 函数

函数是代码复用和模块化的基本单位：

```
int add(int x, int y)
{
    return x + y;
}
```

```
int main()
{
    int sum = add(3, 4);
    printf("sum = %d\n", sum);
    return 0;
}
```

库，其实就是一大堆别人已经写好的函数。

8. 数组

数组是一段连续的、存放同类型数据的内存空间，通过下标访问：

```
int arr[5] = {1, 2, 3, 4, 5};
for (int i = 0; i < 5; i++)
{
    printf("arr[%d] = %d\n", i, arr[i]);
}
```

下标从 0 开始，常用于缓存、批量采集数据。

9. 指针

指针是 C 语言最强大的特性，也是理解嵌入式硬件的关键。

指针，就是**存放地址的变量**：

```
int a = 10;
int *p = &a; // &a 取 a 的地址，p 指向 a
printf("a = %d, *p = %d\n", a, *p); // *p 解引用，得到 p 指向的值
```

- &：取地址；
- *：解引用（通过地址访问那个值）。

在嵌入式里，寄存器就是地址，指针就是操作寄存器的那只手。理解了“指针 = 地址”，你就理解了“库为什么能控制硬件”。

10. 结构体

结构体把不同类型的数据打包成一个整体：

```
struct Student
{
    char name[20];
    int age;
};
```

```
struct Student s = {"Tom", 18};  
printf("%s %d\n", s.name, s.age);
```

- 用 `.` 访问结构体变量的成员；
- 用 `->` 访问结构体指针的成员（等价于 `(*p).成员`）：

```
struct Student *p = &s;  
p->age = 20;      // 等价于 (*p).age = 20
```

嵌入式里的外设库，几乎都是用结构体来组织一个外设的诸多寄存器——后面你看到的库函数参数里，就有大量结构体。

11. 枚举

枚举给一组整数起名字，让代码更可读：

```
enum Week { Mon, Tue, Wed, Thu, Fri, Sat, Sun };  
enum Week today = Fri;    // today 实际是整数 4
```

常用于状态、事件类型、错误码等场景。

12. 联合体

联合体让多个成员共用同一块内存，同一时刻只能存其中一个：

```
union Data  
{  
    int i;  
    float f;  
    char str[20];  
};  
  
union Data data;  
data.i = 10;  
printf("data.i = %d\n", data.i);
```

大小取决于最大的成员，常用于协议解析、数据转换、寄存器映射等。

13. 宏与预处理

宏由 `#define` 定义，在**预处理阶段**（编译之前）做简单的“文本替换”。

定义常量

```
#define LED1_PIN    (E2)    // 之后代码里的 LED1_PIN 都会被替换成 (E2)
```

定义函数式宏

```
#define MAX(a, b)    ((a) > (b) ? (a) : (b))
```

注意：宏只是“原样替换”，所以参数和整个宏体都要用括号包起来，否则容易算错。

条件编译

```
#ifdef APP_RECOGNIZE_USE_CLEAR
    // 这段代码只在定义了这个宏时才被编译
#endif
```

这正好呼应了前言里的那个真实案例——一个 `#define` 就能让整个识别模块“不参与编译”。理解了宏和预处理，你就能看懂这类“隐藏开关”。

除了 `#define`，预处理指令还有 `#include`（包含头文件）和 `#ifdef` / `#ifndef` / `#endif`（条件编译）等。

以上这些，就是我们要用到的大部分 C 语言。先有个印象即可，真正记住它们，靠的是后面不断的动手。

第三部分：认识库文件

这一部分，我们先把开发环境搭好，再看清这个库工程到底长什么样、怎么用。

我们使用的，是逐飞科技（SEEKFREE）开源的 CH32V307 库，配合沁恒（WCH）官方 SDK，开发环境是 MounRiver Studio 2（山河）。

SDK：芯片厂商提供的官方底层代码库。

1. 准备开发环境

在理解库之前，先把开发环境搭起来。本文档使用沁恒 CH32V307 + MounRiver Studio 2（山河）开发，按下面五步走完，你就能打开工程、编译、烧录。

第一步：安装 MounRiver Studio 2（山河）

IDE 是集成开发环境，集成了代码编辑、编译、调试等工具。CH32V307 用的 IDE 是 MounRiver Studio。

IDE（集成开发环境）：写代码 + 编译 + 烧录，都在同一个软件里完成。**MounRiver Studio 2（山河）**：我们本次要安装、使用的 IDE。**编译器 / 调试器**：IDE 里的两个部件——编译器把代码翻译成芯片能执行的形式，调试器让你能单步运行、查看变量的值。

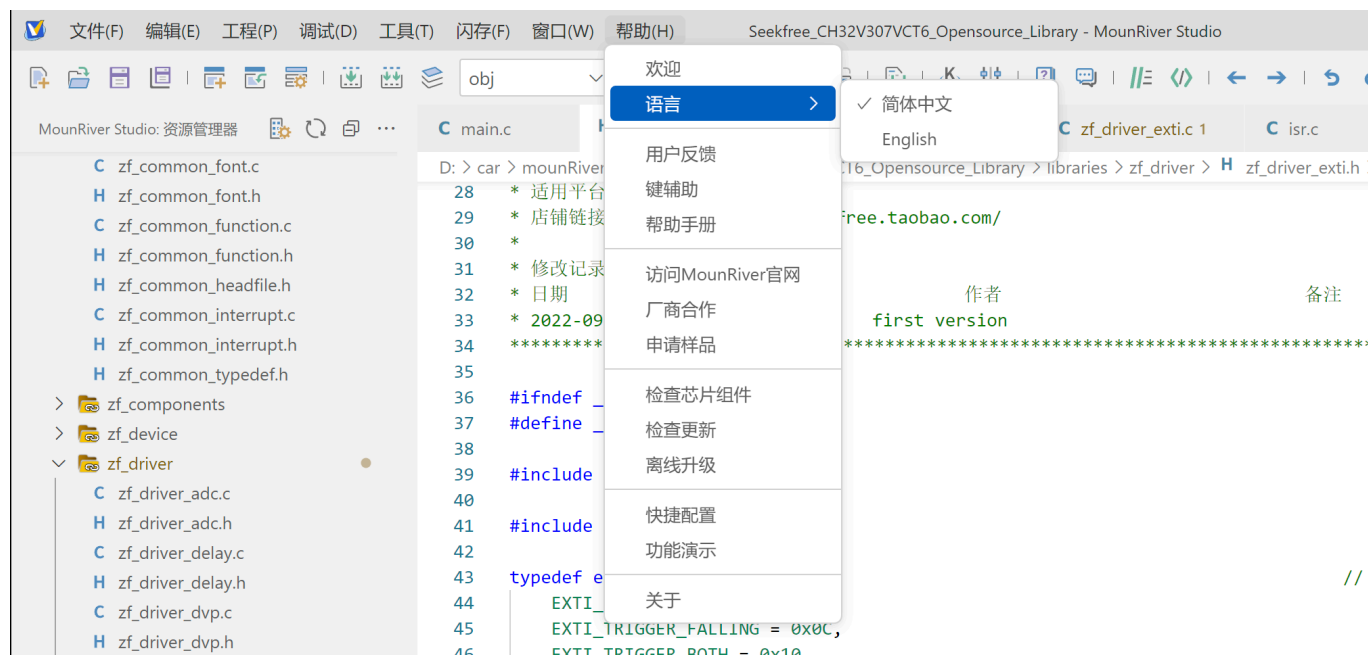
打开官网 [MounRiver](#)：



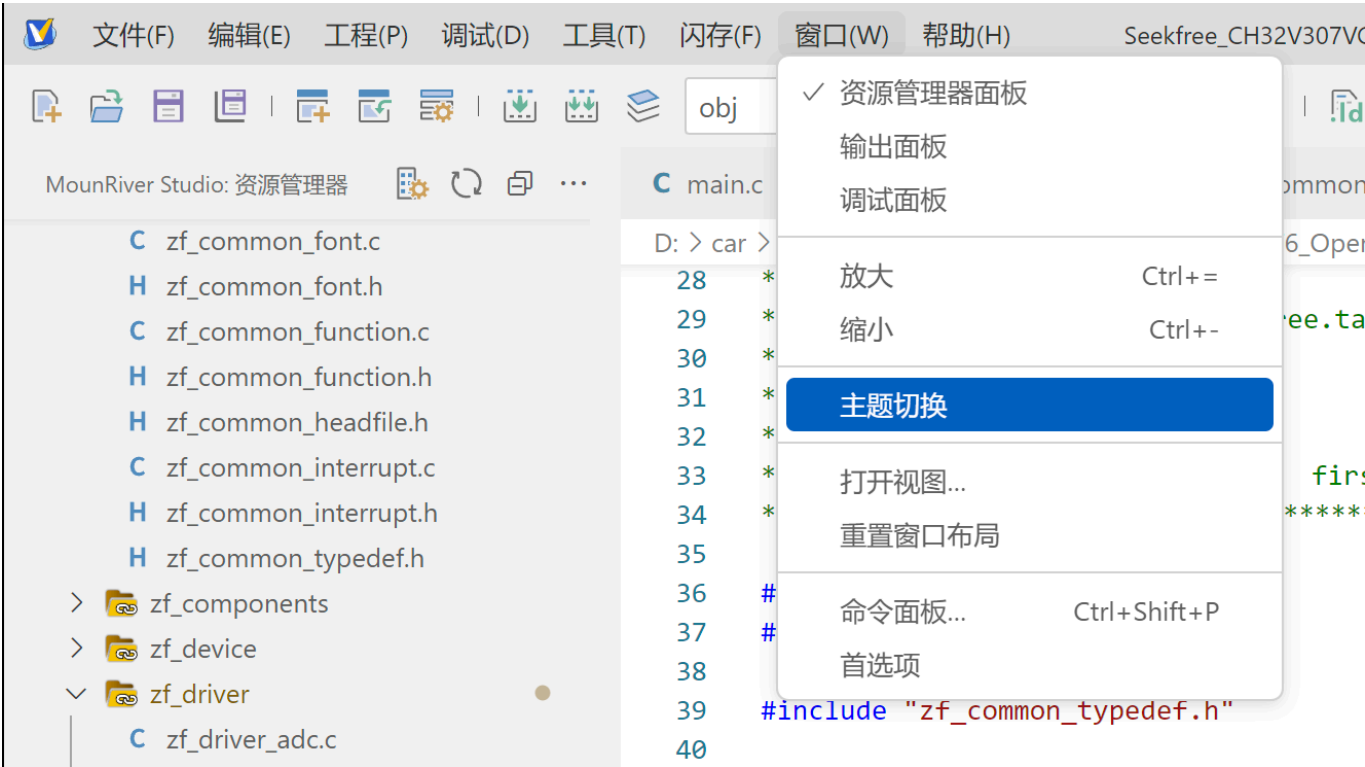
如果页面是英文，点右上角切成中文，再点「下载」栏。点击蓝色的 [MounRiver_Studio_Setup_V220.zip](#) 下载安装包，解压后双击 [MounRiver_Studio_Setup_V220.exe](#) 按提示安装。

第二步：配置界面

装好后打开 IDE，弹窗和驱动一律同意、安装，期间可能会卡顿或重启，等它安静下来即可。然后把界面语言切成中文：



如果觉得白色主题刺眼，可以换个主题：



第三步：获取逐飞开源库

打开 Gitee，先注册账号，搜索 seekfree，进入逐飞的开源工程：



选择仓库：



在右侧快速搜索栏搜 ch32v307，进入对应仓库：



点「克隆/下载」，可以用 git 克隆，也可以直接下载 zip 后解压。注意：**Example** 目录下的例程**不是完整工程**，**Seekfree_CH32V307VCT6_Opensource_Library** 目录才是完整的开源库。

仓库 / 克隆 / git / zip：仓库是代码在网上的存放处；“克隆”是用 git 工具把仓库完整复制到本地，也可以直接下载 zip 压缩包再解压。

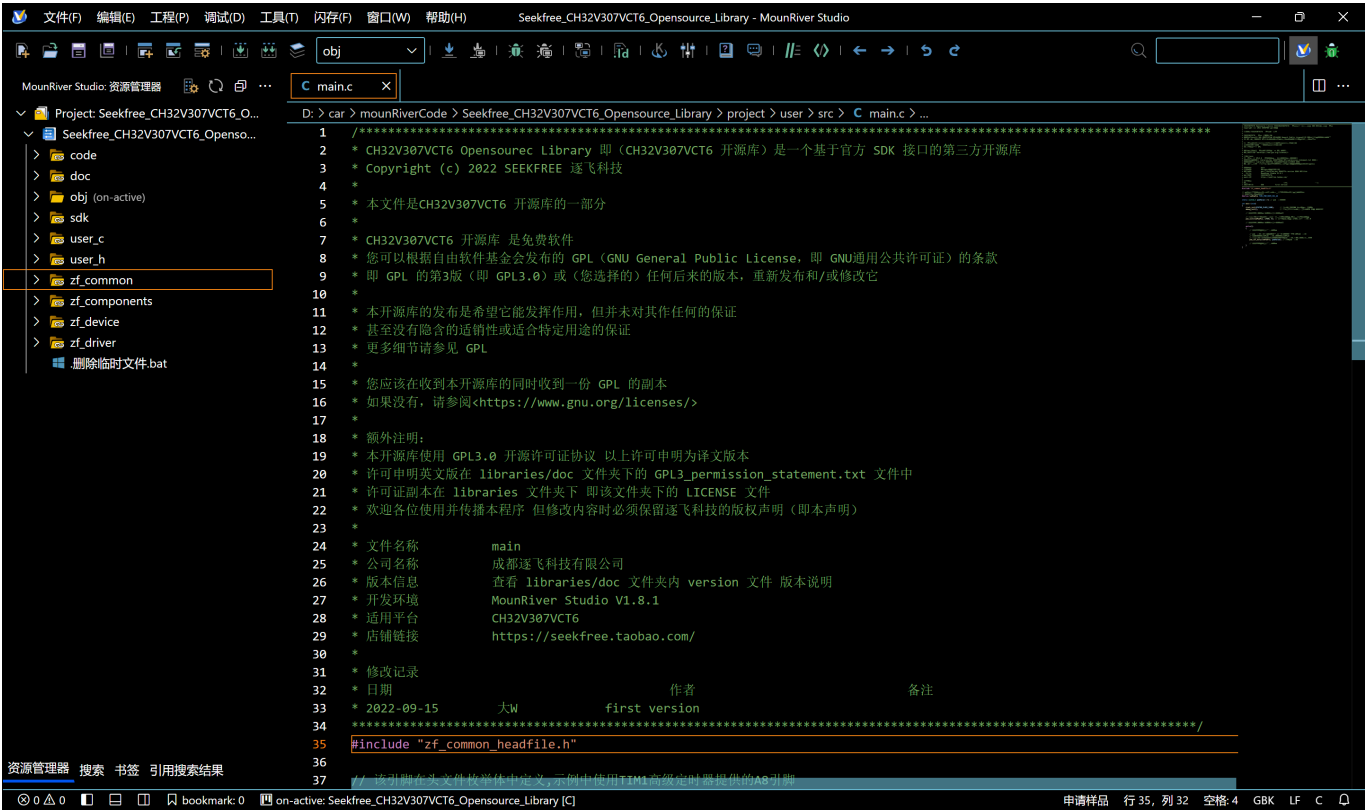
第四步：打开开源库

把 **Seekfree_CH32V307VCT6_Opensource_Library** 文件夹复制到你打算存放工程的地方，进入 **Seekfree_CH32V307VCT6_Opensource_Library\project\mrs**，双击 **Seekfree_CH32V307VCT6_Opensource_Library.wvproj** 即可打开工程。

工程 / 项目 (.wvproj)：一整套代码的集合。**.wvproj** 是 MounRiver 的工程文件，双击它就能打开整个工程。

第五步：编辑、编译与下载

打开工程后是下面这个界面——左侧是工程目录树，中间是代码编辑区：



顶部工具栏有三个常用按钮，前两个是「编译」和「重新编译」：



- 编译：只编译改过的文件，省时间；
- 重新编译：编译所有文件，保证都是最新的。

第三个是「下载」按钮，把编译生成的固件烧进单片机：



重新编译：把全部文件重新编一遍，确保都是最新的（只编译改过的文件叫“编译”）。**固件**：编译的产物，也就是要烧进芯片的那个文件。

2. 工程目录总览

```
CHV307_Library-master/
├── Example/
│   ├── E01_gpio_demo/      ...  E12_usb_cdc_demo/
│   └── libraries/
├── Seekfree_CH32V307VCT6_Opensource_Library/
├── 【封装】核心板整体原理图+PCB封装/
└── 【文档】说明书 芯片手册等/
```

12 个例程

例程共用的库

逐飞开源库工程

我们真正要关注的，是 **Example** 里的 12 个例程，以及它们共用的一套 **libraries** 库。

3. 逐飞库的分层

打开 **libraries** 文件夹，里面是这样的分层：

```
libraries/
├── doc/           # 文档：开源许可证、版本说明
├── sdk/           # 沁恒官方 SDK（最底层，直接操作寄存器）
│   ├── Core/     # RISC-V 内核支持
│   ├── Ld/       # 链接脚本
│   ├── Peripheral/ # 标准外设库：ch32v30x_gpio.c 等
│   └── Startup/   # 启动文件
├── zf_common/    # 逐飞通用封装：时钟/调试/fifo/字体/中断 + 总头文件
├── zf_driver/    # 逐飞底层驱动：软件 IIC、软 SPI
├── zf_device/    # 逐飞器件驱动：OLED/TFT/IMU/电机驱动/按键/摄像头...
└── zf_components/ # 逐飞组件：逐飞助手（上位机调试接口）
```

用一句话理解这个分层：

sdk 在最底层，直接和寄存器打交道；**zf_common / zf_driver** 在中间做通用封装；**zf_device / zf_components** 是最上层，面向具体器件和功能。

驱动 / 封装 / 分层：驱动是直接操作硬件的代码；封装是把复杂的底层操作包成好用的函数；分层是把它们按“底层 → 上层”组织起来。

对初学者来说，最重要的是记住一个文件：

zf_common_headfile.h——逐飞把所有头文件都汇总进了这一个“总头文件”。你在自己的代码里，通常只需要 `#include "zf_common_headfile.h"` 这一句，就能用上库里的绝大多数函数。

总头文件 zf_common_headfile.h：逐飞把库里所有头文件汇总成的唯一入口，只 `#include` 它一个就能用全库。

4. 读懂第一个例程：E01_gpio_demo

先认识 GPIO。**GPIO**（General Purpose Input/Output，通用输入输出口）是单片机最基本、最常用的外设：软件可以直接把某个引脚的电平拉高或拉低（**输出**），也可以读取某个引脚当前是高还是低（**输入**）。点亮 LED、读按键，靠的都是它。

GPIO：通用输入输出口，软件可以直接控制引脚的电平高低。**引脚 / 电平**：引脚是芯片伸出来的“脚”；电平就是它上面的电压高低，用高电平（1）和低电平（0）表示。

用 GPIO 之前，要先告诉芯片“这个引脚是输入还是输出、用哪种模式”——输出常用**推挽输出**（能主动拉高/拉低，点灯就用它），输入常用**上拉/下拉输入**（读按键时保证电平确定）。这些配置都由库函数代劳，你只需传对参数。

推挽输出 / 上拉、下拉输入：GPIO 的几种工作模式——推挽输出能主动把引脚拉高或拉低；上拉/下拉输入则在引脚没接信号时，给它一个确定的默认电平。

打开 `Example/E01_gpio_demo/user/src/main.c`，这是我们的第一个例程——**点亮 LED**。

它只有短短几十行，却包含了嵌入式程序几乎全部的骨架：

```
#include "zf_common_headfile.h"           // 1. 引入库的总头文件

#define LED1_PIN    (E2)                  // LED 接在 E2 引脚
#define LED2_PIN    (E9)                  // LED 接在 E9 引脚

int main(void)
{
    clock_init(SYSTEM_CLOCK_120M);        // 2. 初始化系统时钟 120MHz
    debug_init();                          // 3. 初始化调试（务必保留）

    gpio_init(LED1_PIN, GPIO, 0, GPIO_PUSH_PULL); // 4. 初始化 GPIO 为推挽输出
    gpio_init(LED2_PIN, GPIO, 0, GPIO_PUSH_PULL);

    while(1)                               // 5. 主循环：永远执行
    {
        gpio_set_level(LED1_PIN, 1);       // 引脚输出高电平 → LED 亮
        system_delay_ms(100);              // 延时 100 毫秒
        gpio_set_level(LED1_PIN, 0);       // 引脚输出低电平 → LED 灭
        system_delay_ms(100);

        gpio_toggle_level(LED2_PIN);       // 翻转电平：亮变灭、灭变亮
        system_delay_ms(100);
    }
}
```

原例程里还有一段读取输入引脚 B2 的代码，这里先略去，打开文件时你会看到。

把它拆成四步，就是所有嵌入式程序的通用骨架：

① 初始化时钟 → ② 初始化调试 → ③ 初始化外设 → ④ 主循环

- `clock_init`：让芯片以设定频率运行；
- `debug_init`：初始化调试串口，务必保留；
- `gpio_init`：把某个引脚配置成输入或输出；
- `while(1)`：程序在这里无限循环，做你让它反复做的事。

循环里用到的几个 GPIO 函数，含义也很直白：

- `gpio_set_level(pin, 1)` —— 设引脚为高电平（点灯）；
- `gpio_set_level(pin, 0)` —— 设引脚为低电平（灭灯）；
- `gpio_toggle_level(pin)` —— 翻转电平；
- `system_delay_ms(n)` —— 阻塞延时 n 毫秒。

注意 `system_delay_ms` 是**阻塞**延时：这期间 CPU 在空转等时间，什么别的活都干不了。这个“局限”先记在心里，以后学到定时器和中断时会专门解决它。

阻塞延时：让 CPU 原地空等指定时间，这期间它什么别的活都干不了。

顺带一提：`#define LED1_PIN (E2)` 里的 `E2`，指的是引脚编号（E 组的第 2 号引脚）。引脚分配可以查工程里的 `第十七届 智能车推荐引脚分配.txt`，你烧录时要用到的就是这些引脚。

引脚编号 (E2、A8...)：引脚的“名字”——字母代表引脚组，数字是组内编号，具体见引脚分配表。

5. PWM：让灯“呼吸”起来

考核里的“呼吸灯”，本质是让 LED 的亮度**平滑变化**。但直接操作 GPIO 只能让灯“亮”或“灭”，做不出中间亮度，这时就要用到 PWM。

****PWM（脉宽调制）****的思路是：让引脚以很高的频率快速开关，通过调节一个周期里“高电平所占的比例”（**占空比**），等效出一个可变的平均电压/亮度：

占空比 100%：一直高电平 → 最亮
占空比 50%：一半高一半低 → 中等亮度
占空比 0%：一直低电平 → 熄灭

两个关键参数：

- **频率（周期）：**一个完整波形的时间长度，由定时器决定；
- **占空比：**一个周期内高电平所占的比例。

PWM（脉宽调制）：让引脚高频快速开关，用高电平所占的比例来等效出不同的平均电压 / 亮度。 **占空比 / 频率 / 周期：**占空比是一个周期里高电平所占的比例；频率是每秒完成的周期数；周期是一个完整波形的时间长度。

在 CH32V307 上，PWM 由**定时器硬件**自动产生，CPU 不用空转。逐飞库把接口封装在 `zf_driver_pwm.h` 里，主要就两个函数：

硬件 PWM / 定时器：由定时器这个外设自动产生 PWM 波形，不占用 CPU 的时间。

```
#define ledPwmPin  TIM1_PWM_MAP0_CH1_A8    // 引脚用库里的枚举表示

pwm_init(ledPwmPin, 17000, 0);             // 初始化：引脚、频率 17KHz、初始占空比 0
pwm_set_duty(ledPwmPin, pwmValue);         // 设置占空比：0 ~ 10000 对应 0% ~ 100%
```

例程 `E04_pwm_demo` 就是 PWM 的官方参考。做呼吸灯时，只要在主循环里让 `pwmValue` 从 0 渐变到 10000、再渐变回 0，灯就会“由亮到暗、由暗到亮”地呼吸起来。

6. 中断：让程序能“被打断”

主循环里的代码是一条条顺序执行的，但有些事必须“一发生就立刻处理”——比如按键一按下就要响应。这时就要用到**中断**。

中断的意思是：当某个事件发生时（引脚电平变化、定时器到时间.....），CPU 暂停手头的代码，跳去执行一段专门的**中断服务函数**，处理完再回到刚才的位置继续：

中断 / 中断服务函数：有事发生时“打断”主循环去处理的机制；被跳转去执行的那段代码，就叫中断服务函数。

主循环顺序执行
↓ 事件发生，触发中断
跳进中断服务函数
↓ 处理完
回到刚才的位置继续

逐飞库把外部中断封装在 `zf_driver_exti.h` 里，核心就一个函数：

```
exti_init(KEY_PIN, EXTI_TRIGGER_FALLING); // 初始化：中断引脚 + 触发方式
```

触发方式有三种，对应电平变化的方向：

- `EXTI_TRIGGER_RISING` —— 上升沿（低变高）；
- `EXTI_TRIGGER_FALLING` —— 下降沿（高变低，按键按下时常用）；
- `EXTI_TRIGGER_BOTH` —— 双边沿（任何变化都触发）。

上升沿 / 下降沿 / 双边沿：引脚电平“由低变高 / 由高变低 / 任意变化”的那一刻，用作中断的触发条件。

初始化之后，还要在工程的 `isr.c` 里找到对应的**中断服务函数**（形如 `EXTI15_10_IRQHandler`），在函数里写你要做的事——比如翻转一个 LED。

例程 `E06_exti_demo` 是外部中断的官方参考。要实现“按键控制 LED”，照它改即可：按下触发中断，在中断服务函数里翻转 LED，就能“按一下亮、再按一下灭”。

本次考核（呼吸灯 / 呼吸闪烁交替）**不涉及中断**，这一节先建立概念即可；后续培训会专门展开按键、定时器中断等内容。

第四部分：考核任务

学完上面的内容，现在轮到你动手了。本次培训的考核任务如下。

基础任务：呼吸灯

让板载的 LED 产生**肉眼可见、平滑**的呼吸效果：

从亮 → 到暗 → 从暗 → 到亮 → 再变暗

要求：

- LED 周期性往复变化；

- 变化过程中**没有可感知的跳变或频闪**；
- 呼吸周期长短**不作硬性规定**，由你自己调节，觉得自然即可。

实现口径（两种都算达标）：

- **GPIO + 阻塞延时**，用软件方式模拟（最容易上手）；
- **定时器硬件 PWM** 实现者，验收时**予以加分肯定**。

例程参考：软件方式看 [E01_gpio_demo](#)，硬件 PWM 看 [E04_pwm_demo](#)。

提高任务：呼吸与闪烁交替

在一个呼吸周期之后，紧接着一个闪烁周期，如此交替循环：

呼吸 → 闪烁 → 呼吸 → 闪烁 →

闪烁要求：

- 每 **0.5 秒** 切换一次亮暗；
- 一个闪烁周期按“亮 → 暗 → 亮 → 暗 → 亮 → 暗”变化一轮，共 **3 秒**；
- 之后又是一个呼吸周期，周而复始。

这个任务真正考察的，是你对“程序运行状态”的描述与切换组织能力——什么时候进入呼吸、什么时候进入闪烁，怎么把这两个状态干净地接起来。

例程参考：在 [E01_gpio_demo](#)（闪烁）和 [E04_pwm_demo](#)（呼吸）的基础上组合即可。

验收方式

- 现场演示，负责人当场验收；
- 考核周期：**1 周**；
- 负责人：**李政熙**。

写在最后

从第一颗 LED 被点亮开始，你就在真正地“和硬件对话”了。

不用急着一次全部理解——先跑起来，再回头看今天的内容，很多话会自然变得清晰。

遇到问题，大胆问、大胆查、大胆和 AI 交流。

祝大家顺利点亮第一颗灯。